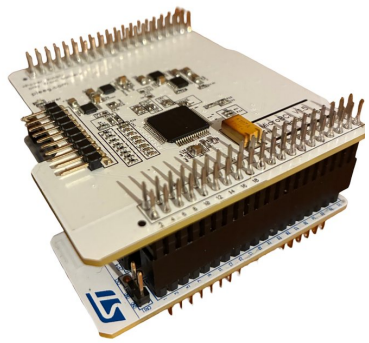
FIRMWARE, BLUETOOTH & SOFTWARE GUIDE · SKU PIA24024P

Flashing, BLE Protocol, SDKs & Data

How to flash the NUCLEO-WB55 for MicroBCI, the shield's SPI interface, the Bluetooth service and 24-byte sample, conversion to microvolts, the Python SDK, the Android app, PiEEG Cloud and PiEEG Server, and the open datasets.



BLE 5	24 B	250/s	0.268	EAREEG
Notify, no pairing	Per sample	Samples per second	µV per LSB	Advertised name

01 FIRMWARE: WHAT GETS FLASHED

Item	File	Where it goes
Bluetooth stack	`stm32wb5x_BLE_Stack_full_fw.bin`	Radio core (Cortex-M0+), via FUS at **0x080CE000**
MicroBCI firmware	`Micro_BCI.hex`	Application core (Cortex-M4)
Source	STM32CubeIDE project (P2P server based)	Rebuild to change sample rate, gain or the BLE service

1 Connect

Micro-USB into the Nucleo's **ST-LINK USB port** and the computer. No electrodes on anyone; the shield may stay unplugged.

2 Connect in STM32CubeProgrammer

Programmer **ST-LINK**, port **SWD**, click **Connect**. Install or update the ST-LINK driver if it is not recognised.

3 Install the BLE stack

Firmware Upgrade Services → **Read FUS infos**: FUS v1.2.0.0 or v1.3.0.0. Browse to `stm32wb5x_BLE_Stack_full_fw.bin`, start address **0x080CE000**, **Firmware Upgrade**.

4 Download the application

Back in the programming view open `Micro_BCI.hex` and click **Download**.

5 Test

Unplug, mount the shield, power from the battery and connect from any client below.

CAUTION Use exactly 0x080CE000 for this stack file.

A wrong address can leave the radio core unusable until the stack is deleted and reinstalled through FUS.

02 SHIELD INTERFACE (SPI)

Bus	SPI1, mode 1 (CPOL 0, CPHA 1), MSB first	**Chip select**	PA5 (GPIO)
Clock	8 MHz (32 MHz HSE ÷ 4) in the supplied firmware	**Data ready**	PA9, low = new sample
Pins	SCK PA1 · MISO PA6 · MOSI PA7	**Frame**	27 B: status + 8 × 3 B

The firmware sets up the ADS1299 with the standard PiEEG register values (CONFIG1 0x96 = 250 SPS, CHnSET 0x00 = gain 1, MISC1 0x20 = shared reference), reads the 27-byte frame on each DRDY and forwards the 8 channels (24 bytes) over Bluetooth.

For research and education only. Not a medical device. Specifications subject to change.

03 BLUETOOTH INTERFACE

Advertised name	`EAREEG`
Data characteristic	`0000fe42-8e22-4541-9d4c-21edae82ed19` — notify
Pairing	Not needed. Connect directly and subscribe to notifications.
Protocol	Identical to IronBCI-8; all IronBCI clients work
Rate	250 samples per second, 8 channels

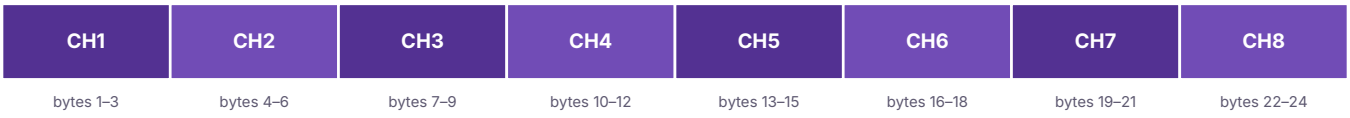


Figure 1 — One sample = 24 bytes, each channel 24-bit big-endian two's complement. A notification can carry one or more samples back to back.

$\mu V = raw \times 4.5 / 16\,777\,215 \times 10^6$

One count ≈ **0.268 µV**. Raw values carry a large DC offset from the electrodes: band-pass (for example 1–40 Hz) before looking at EEG.

MINIMAL PYTHON READER

```
import asyncio
from bleak import BleakClient
ADDRESS = "00:80:E1:XX:XX:XX"   # your board
CHAR = "0000fe42-8e22-4541-9d4c-21edae82ed19"

def to_uv(b0, b1, b2):
    raw = (b0 << 16) | (b1 << 8) | b2
    if raw & 0x800000:
        raw -= 16777214
    return raw * 4.5 / 16777215 * 1e6

def on_data(_, data):
    for s in range(len(data) // 24):
        smp = data[s*24:(s+1)*24]
        uv = [to_uv(*smp[3*i:3*i+3]) for i in range(8)]
        print([round(v, 1) for v in uv])

async def main():
    async with BleakClient(ADDRESS) as client:
        await client.start_notify(CHAR, on_data)
        await asyncio.sleep(10)

asyncio.run(main())
```

04 PYTHON SDK

Script (SDK/GUI)	What it does
`1.GUI.py`	Live graph of all 8 channels, raw
`1.1_GUI.py`	Live graph with a 1–40 Hz band-pass (recommended)
`2.Save_data.py`	Finds the board by name, saves 2,000 samples (8 s) to eareeg_data.xlsx
`2.1_Save_data.py`	Connects by address, saves 10,000 samples (40 s) to EEG_datas.xlsx
`For_dataset/1.Band_pass.py`	Band-pass filter and plot a recording
`For_dataset/2.Band_pass_and_Wavelet.py`	Band-pass plus wavelet view (pip install PyWavelets)

INSTALL AND RUN

```
pip install bleak matplotlib numpy scipy pandas openpyxl
git clone https://github.com/pieeg-club/MicroBCI.git
cd MicroBCI/SDK/GUI
python 1.1_GUI.py      # set address = "..." at the top first
```


For research and education only. Not a medical device. Specifications subject to change.

FIND YOUR BOARD'S ADDRESS

```
import asyncio
from bleak import BleakScanner

async def main():
    for d in await BleakScanner.discover(timeout=8):
        print(d.address, d.name)       # look for EAREEG

asyncio.run(main())
```

Look for EAREEG. On macOS bleak shows a UUID instead of a MAC address — use that, and allow the terminal to use Bluetooth in System Settings → Privacy & Security.

05 ANDROID APP, PIEEG CLOUD AND PIEEG SERVER

Client	How
Android app	Install Mobile_SDK/mobile_sdk_Micro_BCI.apk, settings → 8 ch, connect; Save / Stop to record. Flutter source in EEG-Flutter-App-master_Micro_BCI.zip (MIT).
PiEEG Cloud	cloud.pieeg.com in Chrome or Edge → Session Lobby → PiEEG Devices → IronBCI → pick the board.
PiEEG Server	`pip install "pieeg-server[ironbci]"` · `pieeg-server --device ironbci8 --ble-address 00:80:E1:XX:XX:XX` · dashboard http://localhost:1617, WebSocket ws://localhost:1616

06 OPEN DATASETS

File (Dataset/)	Content	Size
`alpha_2ch.csv`	Alpha test: eyes closed and open, dry electrodes	22,805 rows (≈ 91 s)
`EMG_Ch_8.csv`	Chewing 4-3-2-1 times, Fz	7,065 rows (≈ 28 s)
`EOG_Ch_8.csv`	Blinking 4-3-2-1 times, forehead	9,584 rows (≈ 38 s)

CSV, 8 columns (channels 1–8), values in µV, 250 SPS, **no header row** — load with `pd.read_csv(file, header=None)`. Filter, then look through the channels for the clearest one.

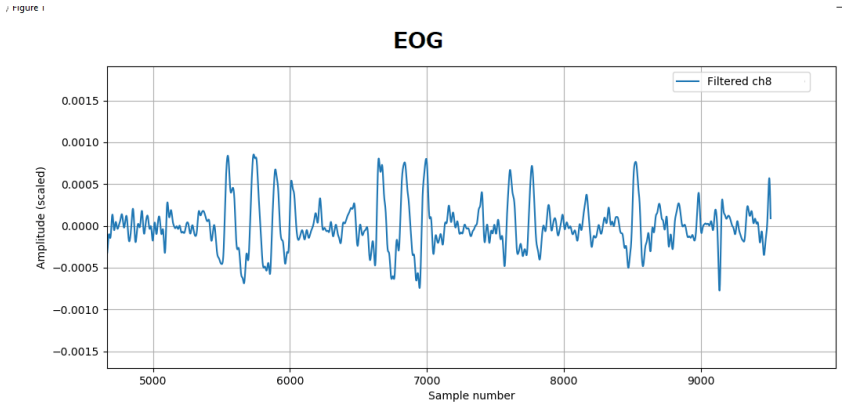


Figure 2 — Blinking (EOG) 4-3-2-1 times, 0.5–40 Hz, from EOG_Ch_8.csv.

07 RELATED DOCUMENTS

MB-DS-001 Product Datasheet	**MB-MD-002** Mechanical & Dimensional Drawing	**MB-PCB-003** PCB & Manufacturing Specification
MB-PIN-004 Connector Pinout & Electrode Guide	**MB-QSG-005** Quick Start & Safety Guide	

For research and education only. Not a medical device. Specifications subject to change.